JavaScript is disabled on your browser.

compact1, compact2, compact3

java.nio.channels

Interface ScatteringByteChannel

All Superinterfaces:

AutoCloseable, Channel, Closeable, ReadableByteChannel

All Known Implementing Classes:

DatagramChannel, FileChannel, Pipe.SourceChannel, SocketChannel

---

```
public interface ScatteringByteChannel
extends ReadableByteChannel
```

A channel that can read bytes into a sequence of buffers.

A *scattering* read operation reads, in a single invocation, a sequence of bytes into one or more of a given sequence of buffers. Scattering reads are often useful when implementing network protocols or file formats that, for example, group data into segments consisting of one or more fixed-length headers followed by a variable-length body. Similar *gathering* write operations are defined in the GatheringByteChannel interface.

Since:

1.4

Method Summary

Abstract Methods

Modifier and Type	Method	Description
long	**read**(**ByteBuffer**[] dsts)	Reads a sequence of bytes from this channel into the given buffers.

long	**read**(**ByteBuffer**[] dsts, int offset, int length)	Reads a sequence of bytes from this channel into a subsequence of the given buffers.

Methods inherited from interface java.nio.channels.ReadableByteChannel

read

Methods inherited from interface java.nio.channels.Channel

close, isOpen

Method Detail

read

```
long read(ByteBuffer[] dsts,
          int offset,
          int length)
   throws IOException
```

Reads a sequence of bytes from this channel into a subsequence of the given buffers.

An invocation of this method attempts to read up to *r* bytes from this channel, where *r* is the total number of bytes remaining the specified subsequence of the given buffer array, that is,

```
dsts[offset].remaining()
    + dsts[offset+1].remaining()
    + ... + dsts[offset+length-1].remaining()
```

at the moment that this method is invoked.

Suppose that a byte sequence of length *n* is read, where `0` <= *n* <= *r*. Up to the first `dsts[offset].remaining()` bytes of this sequence are transferred into buffer `dsts[offset]`, up to the next `dsts[offset+1].remaining()` bytes are transferred into buffer `dsts[offset+1]`, and so forth, until the entire byte sequence is transferred into the given buffers. As many bytes as possible are transferred into each buffer, hence the final position of each updated buffer, except the last updated buffer, is guaranteed to be equal to that buffer's limit.

This method may be invoked at any time. If another thread has already initiated a read operation upon this channel, however, then an invocation of this method will block until the first operation is complete.

Parameters:

`dsts` - The buffers into which bytes are to be transferred

`offset` - The offset within the buffer array of the first buffer into which bytes are to be transferred; must be non-negative and no larger than `dsts.length`

`length` - The maximum number of buffers to be accessed; must be non-negative and no larger than `dsts.length - offset`

Returns:

The number of bytes read, possibly zero, or -1 if the channel has reached end-of-stream

Throws:

`IndexOutOfBoundsException` - If the preconditions on the offset and length parameters do not hold

`NonReadableChannelException` - If this channel was not opened for reading

`ClosedChannelException` - If this channel is closed

`AsynchronousCloseException` - If another thread closes this channel while the read operation is in progress

`ClosedByInterruptException` - If another thread interrupts the current thread while the read operation is in progress, thereby closing the channel and setting the current thread's interrupt status

`IOException` - If some other I/O error occurs

read

```
long read(ByteBuffer[] dsts)
   throws IOException
```

Reads a sequence of bytes from this channel into the given buffers.

An invocation of this method of the form `c.read(dsts)` behaves in exactly the same manner as the invocation

```
c.read(dsts, 0, dsts.length);
```

Parameters:

`dsts` - The buffers into which bytes are to be transferred

Returns:

The number of bytes read, possibly zero, or -1 if the channel has reached end-of-stream

Throws:

NonReadableChannelException - If this channel was not opened for reading

ClosedChannelException - If this channel is closed

AsynchronousCloseException - If another thread closes this channel while the read operation is in progress

ClosedByInterruptException - If another thread interrupts the current thread while the read operation is in progress, thereby closing the channel and setting the current thread's interrupt status

IOException - If some other I/O error occurs

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.